ФГБОУ ВО «Пензенский государственный университет»
Политехнический институт
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №5
по дисциплине «Логика и ОА в ИЗ»
на тему «Определение характеристик графов»

Выполнили: ст. гр. 25ВВВ2
Перетрухин О. С.
Павлов А. Д.
Проверили:
Юрова О. В.
Деев М. В.

Пенза 2026

Цель работы: изучить представление графа матрицей смежности и способы определения количества рёбер, степеней вершин, изолированных, концевых и доминирующих вершин.

1. Постановка задачи

Задание 1: сгенерировать с помощью генератора случайных чисел матрицу смежности неориентированного графа G, вывести её на экран, определить размер графа и найти изолированные, концевые и доминирующие вершины.

В отчёте рассматривается выполнение задания 1. Задание 2*, связанное с матрицей инцидентности, выполняется по указанию преподавателя и в данный отчёт не включено.

2. Описание программы

Пользователь вводит число вершин N. Для матрицы динамически выделяется память. Главная диагональ заполняется нулями. Для каждой пары i < j генерируется случайное значение 0 или 1, которое записывается одновременно в matrix[i][j] и matrix[j][i]. Это обеспечивает симметрию матрицы неориентированного графа без петель.

Функция countEdges() подсчитывает единицы выше главной диагонали, учитывая каждое ребро один раз. Функция findVerticles() вычисляет степень вершины как сумму элементов строки. После полного просмотра строки признак terminal устанавливается по условию degree == 1. Изолированная вершина не имеет рёбер, а доминирующая смежна со всеми остальными. В конце программы выделенная память освобождается.

Для неориентированного графа матрица смежности должна быть симметричной. Степень вершины равна сумме элементов соответствующей строки: степень 0 соответствует изолированной вершине, степень 1 — концевой. Доминирующая вершина смежна с каждой другой вершиной.

3. Исходный код программы

Листинг 1 — Программа определения характеристик графа

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <time.h>
#include <stdlib.h>

int countEdges(int** matrix, int N) {
    int count = 0;
    for (int i = 0; i < N; i++) {
        for (int j = i + 1; j < N; j++) {
            if (matrix[i][j] == 1) {
                count++;
            }
        }
    }
    return count;
}
void findVerticles(int** matrix, int N) {
    int degree;
    bool isIsolated;
    bool terminal;
    bool dominant;
    for (int i = 0; i < N; i++) {
        degree = 0;
        isIsolated = true;
        terminal = false;
        dominant = true;
        for (int j = 0; j < N; j++) {
            if (matrix[i][j] == 1) {
                degree++;
                isIsolated = false;
            }
            if (matrix[i][j] == 0 && i != j) {
                dominant = false;
            }
        }
        terminal = (degree == 1);
        printf("Вершина %d: степень %d, ", i + 1, degree);
        if (isIsolated) {
            printf("изолированная, ");
        }
        else {
            printf("не изолированная, ");
        }
        if (terminal) {
            printf("терминальная, ");
        }
        else {
            printf("не терминальная, ");

        }
        if (dominant) {
            printf("доминирующая\n");
        }
        else {
            printf("не доминирующая\n");
        }
    }
}

int main() {
    srand(time(0));
    setlocale(LC_ALL, "Russian");
    int N;
    printf("Введите количество вершин графа: ");
    scanf("%d", &N);
    if (N <= 0) {
        printf("Ошибка: количество вершин должно быть положительным.\n");
        return 1;
    }

    int** matrix = (int**)malloc(N * sizeof(int*));
```

```
    if (matrix == NULL) {
        printf("Ошибка: не удалось выделить память для матрицы.\n");
        return 1;
    }
    for (int i = 0; i < N; i++) {
        matrix[i] = (int*)malloc(N * sizeof(int));
        if (matrix[i] == NULL) {
            printf("Ошибка: не удалось выделить память для строки матрицы.\n");
            for (int j = 0; j < i; j++) {
                free(matrix[j]);
            }
            free(matrix);
            return 1;
        }
    }
    srand((unsigned int)time(NULL));
    for (int i = 0; i < N; i++) {
        matrix[i][i] = 0;
        for (int j = i + 1; j < N; j++) {
            matrix[i][j] = rand() % 2;
            matrix[j][i] = matrix[i][j];
        }
    }
    printf("Матрица смежности графа:\n");
    for (int i = 0; i < N; i++) {

        for (int j = 0; j < N; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    printf("Характеристики вершин:\n");
    findVerticles(matrix, N);
    int edgeCount = countEdges(matrix, N);
    printf("Количество рёбер в графе: %d\n", edgeCount);
    for (int i = 0; i < N; i++) {
        free(matrix[i]);
    }
    free(matrix);
    return 0;

}
```

4. Результаты контрольного запуска

Программа собрана в Visual Studio 2022, конфигурация Release x64. На рисунке 1 представлен настоящий снимок окна консоли с результатом запуска программы для N = 6. Входное значение указано над выводом программы. Матрица случайная, поэтому при повторном запуске числа могут отличаться.

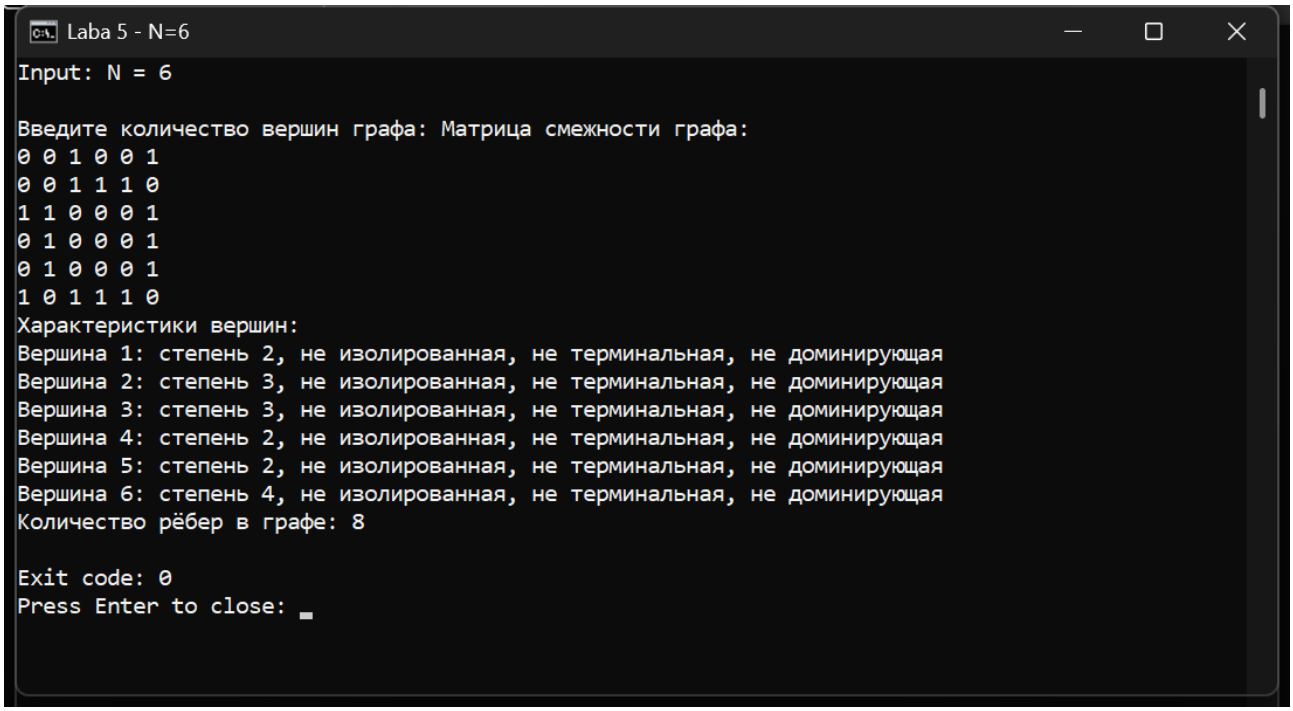


Рисунок 1 — Результат запуска программы при N = 6

Суммы строк полученной матрицы: 2, 3, 3, 2, 2, 4. Число единиц выше главной диагонали равно 8; именно это значение программа вывела как количество рёбер.

5. Анализ результатов

Матрица симметрична, диагональные элементы равны нулю. Сумма степеней вершин равна 16, что совпадает с удвоенным количеством рёбер: 2 × 8 = 16. Следовательно, каждое ребро учтено один раз.

Концевые вершины в данном запуске: отсутствуют. Программа отмечает как терминальные только вершины с итоговой степенью 1; вершины большей степени получают отметку «не терминальная».

6. Дополнительные проверки

При N = 1 программа создаёт матрицу из одного нулевого элемента. Единственная вершина имеет степень 0 и отмечается как изолированная. Она также отмечается как доминирующая: других вершин, с которыми требовалась бы смежность, нет.

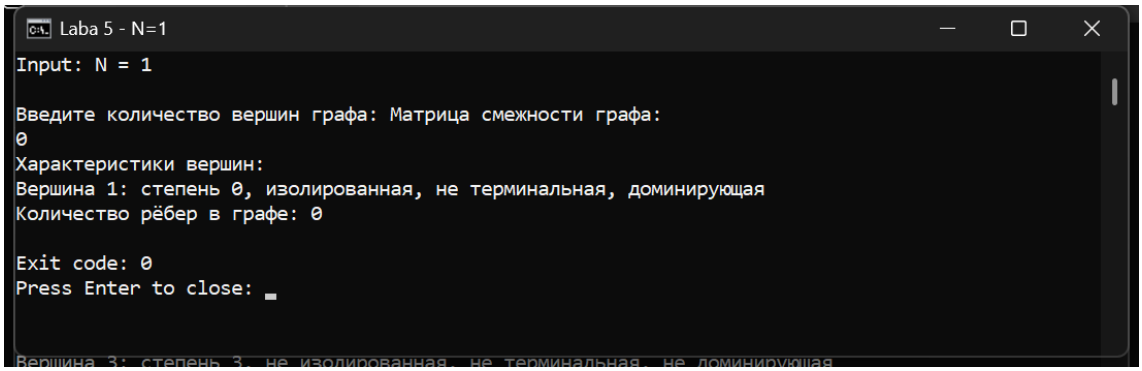


Рисунок 2 — Результат запуска программы при N = 1

При N = 0 программа выводит сообщение об ошибке и завершается с кодом 1 до выделения памяти.

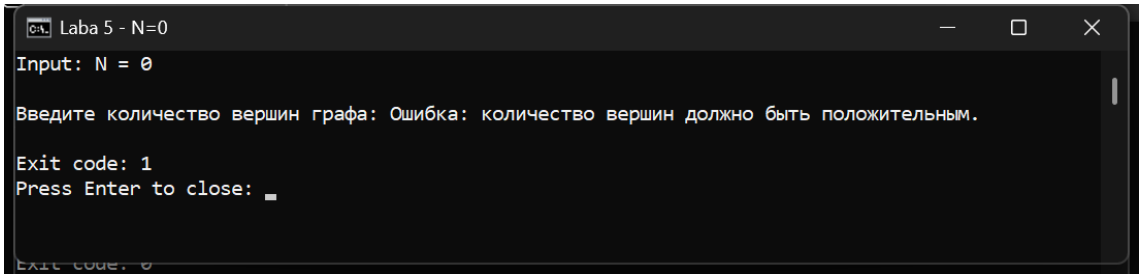


Рисунок 3 — Проверка недопустимого количества вершин N = 0

Вывод: выполнено задание 1: построена симметричная матрица смежности неориентированного графа без петель; определены количество рёбер, степени и типы вершин. Проверки подтвердили равенство суммы степеней удвоенному количеству рёбер и правильность классификации вершин. Дополнительно проверены N = 1 и отклонение N = 0. Задание 2* с матрицей инцидентности в данной версии не реализовано.